

CP++

Detailed Function and API Reference

A row-by-row reference manual for CP++ syntax, functions, methods, examples, and complexity

Version: conversational draft v0.3 expanded reference

Format: each feature is shown as Feature / Example / Explanation / Complexity.

Scope: this document summarizes the CP++ language design defined in conversation, including I/O, types, operators, containers, data structures, testing, and design tradeoffs.

Reading rule: A star such as $O(1)^*$ means amortized or expected. Hash-based operations are expected $O(1)$ unless collisions or adversarial hashing dominate. Container operations can become expensive when they copy, print, slice, materialize, or traverse values.

Contents

1. 1. Program Model, Source Structure, and Basic I/O
2. 2. Declarations, Primitive Types, Defaults, and Conversions
3. 3. Operators, Arithmetic, Comparison, and Bit Operations
4. 4. Control Flow, Ranges, Iteration, and Comprehensions
5. 5. Functions, Pass Modes, Pointers, and References
6. 6. Object Model: Pair, Struct, and Class
7. 7. Common Container Protocols and Literals
8. 8. List, Arr, String, and Bitvector
9. 9. Set, Map, Multiset, HashSet, and HashMap
10. 10. Heap, Stack, Queue, and Deque
11. 11. LinkedList and Node Handles
12. 12. Graph and WeightedGraph
13. 13. Sorting, Comparison, and Selection Helpers
14. 14. Math, Number Theory, Randomness, FFT, and Constants
15. 15. Standard Algorithmic Data Structures
16. 16. Testing and Stress-Testing Framework
17. 17. Example Idioms and Short Programs
18. 18. Non-Goals, Tradeoffs, and Open Questions

How to Read the Detailed API Tables

Each row follows the reference-card style requested: a feature or method, a small C++ example, a detailed explanation, and a complexity or caveat. The examples are not exhaustive; they show the intended normal use. The explanations are deliberately more verbose than a compact cheat sheet so the PDF can work as a design document and not just a syntax list.

Example row style expanded into code

```
List<int> L;  
L.add(5);    // add 5 to the end of L  
int x = L[-1]; // get last element  
L.remove();  // remove and return last element
```

Design note: C++ compresses boilerplate, not algorithmic responsibility. If an operation is $O(n)$, the short syntax does not make it $O(1)$.

1. Program Model, Source Structure, and Basic I/O

CP++ programs are designed to be short, algorithm-first, and implicitly wrapped in the normal C++ contest boilerplate. This section defines top-level execution, printing, reading, flushing, and file redirection.

Feature / function	Example	Detailed explanation	Complexity / notes
top-level statements	<code>print("hello world");</code>	A CP++ source file may contain ordinary statements at the top level. These statements are lowered into an implicit main function. The programmer writes the algorithm, not the include list and fast-I/O boilerplate.	Compile-time lowering
semicolon rule	<code>int x = 5;</code>	Ordinary statements end with semicolons. Blocks use braces like C++: <code>if(condition) { ... }</code> . The language intentionally keeps braces because contest code is frequently pasted, moved, and nested under time pressure.	Syntax rule
<code>print(x)</code>	<code>print(ans);</code>	Prints <code>x</code> followed by a newline. <code>x</code> may be a primitive value, string, Pair, List, Set, Map, struct, or other printable CP++ value. Printing a container prints its elements in the container iteration order.	$O(\text{size of output})$
<code>print(a, b, c)</code>	<code>print(i, j, dist);</code>	Prints multiple arguments separated by single spaces, then appends a newline. This is the normal way to emit a row of answer values without manually writing spaces.	$O(\text{size of output})$
<code>print(x, flush)</code>	<code>print(query, flush);</code>	Prints <code>x</code> and immediately flushes the output buffer. The marker <code>flush</code> is only valid as the final print argument. This is intended for interactive problems; normal print is buffered for speed.	$O(\text{size of output})$; flush cost
<code>describe(x)</code>	<code>describe(dp[i]);</code>	Debug-style print equivalent to printing the variable name and its value, roughly <code>f"x: {x}"</code> . It is intended for quick local debugging, not polished output.	$O(\text{size of output})$
<code>input()</code>	<code>int x = input();</code>	Reads one whitespace-separated token and converts it to the target type. In <code>int x = input();</code> the target type is <code>int</code> . In <code>string s = input();</code> the target type is <code>string</code> and one token is read.	$O(\text{length of token})$
<code>int a, b, c = input()</code>	<code>int a, b, c = input();</code>	Reads three whitespace-separated ints into <code>a</code> , <code>b</code> , and <code>c</code> . Multi-input assignment works when the variables have compatible declared types and the arity is known from the declaration.	$O(\text{number of tokens})$
<code>List<int> L = input()</code>	<code>List<int> L = input();</code>	Reads a line of whitespace-separated integers into a <code>List<int></code> . This is the line-mode list shorthand. It should stop at the newline, not at EOF, unless the implementation explicitly chooses EOF	$O(n)$

		mode.	
List<int> L = input(n)	List<int> a = input(n);	Reads exactly n integers into a List<int>. This is the standard known-length input form for arrays/vectors.	O(n)
List^2<int> G = input(n, m)	List^2<int> grid = input(n, m);	Reads an n by m rectangular 2D list. This is the standard grid/matrix input form. Higher-dimensional List^k input uses the same idea with more dimensions.	O(product of dimensions)
string s = input(getline)	string line = input(getline);	Reads one full line as a string, excluding the trailing newline. After token input, CP++ should consume the pending newline so this reads the next meaningful line rather than returning an accidental empty string.	O(line length)
set_io_in("file.txt")	set_io_in("paint.in");	Redirects future input() calls to the chosen input file. This is the explicit file-input form for file-based judges and local testing.	O(1) setup
set_io_out("file.txt")	set_io_out("paint.out");	Redirects future print calls to the chosen output file. This is the explicit file-output form.	O(1) setup
set_io("paint")	set_io("paint");	Convenience helper that uses paint.in for input and paint.out for output. This is aimed at USACO-style file naming.	O(1) setup
f"...{expr}..."	string msg = f"ans:{ans}";	Formatted string literal. Expressions inside braces are evaluated and converted to strings using CP++ printing rules. This is for readable debug output and formatted contest output.	O(result length)
{x::2}	print(f"avg:{avg::2}");	Inside an f-string, formats numeric x fixed/rounded to two decimal places. The number after :: is the displayed precision.	O(result length)
{x::%}	print(f"rate:{p::%}");	Inside an f-string, formats x as a percentage, usually by multiplying by 100 and appending %. A precision variant such as {x::2%} may display exactly two decimal places.	O(result length)
{x.round(2)}	print(f"x:{x.round(2)}");	Method-style explicit rounding inside a formatted string. This is more verbose than {x::2} but easier to understand in some contexts.	O(result length)

Usage notes

- The main design decision in the I/O layer is that input is target-typed. This is not how C++ streams work, but it matches how contest input is described: read an int, read a list of n ints, read a line.
- The getline form exists specifically to avoid the classic token-input followed by line-input trap. CP++ should define this behavior clearly so the programmer does not need a cin.ignore-style ritual.
- Formatted strings are not meant to replace every possible formatting library. They are meant to handle the two common contest needs: readable debug strings and fixed-decimal output without stream manipulators.

- File redirection is deliberately simple. It is meant for local testing and USACO-style problems, not for a general filesystem API.

Combined I/O, list input, formatted output

```
set_io("paint");
int n = input();
List<int> a = input(n);
print(F"max:{max(a)}, avg:{{(sum(a)/n)::2}}");
```

2. Declarations, Primitive Types, Defaults, and Conversions

CP++ is statically typed. The primitive names are chosen for competitive-programming intent rather than exact C++ spelling. Defaults are deliberately defined to reduce uninitialized-value bugs in short contest programs.

Feature / function	Example	Detailed explanation	Complexity / notes
bool	bool ok = true;	Boolean type with literal values true and false. It is comparable, with true > false. The default/sentinel value is false.	O(1)
int	int n = 200000;	The default integer type. It represents a 64-bit signed integer, equivalent to C++ long long. CP++ intentionally avoids the common contest bug where C++ int overflows unexpectedly.	O(1)
bigint	bigint x = choose(100,50);	Dynamically sized integer for values beyond 64-bit range. Arithmetic is exact but slower than int. It should appear only when a problem genuinely needs arbitrary precision.	Depends on digit count
char	char c = 'A';	Single character type. The default is null character 0. Characters compare by ASCII/codepoint order unless a later Unicode model is defined.	O(1)
float	float p = 0.125;	High-precision floating type, conceptually C++ + long double. It is used for geometry, probabilities, averages, and numeric output.	O(1)
bigfloat	bigfloat z = (bigfloat)x;	Fixed-point big numeric type with 64 binary fractional places. It exists mainly to resolve collisions between float and bigint. It should almost never be necessary in normal CP.	Depends on size
string	string s = "abc";	String type with char-list-like behavior plus string-specific methods. It supports indexing, slicing, len, comparison, input, and formatted output.	Varies
bitvector	bitvector seen;	Packed dynamic bit array. It has a vector<bool>-like API but is named explicitly so users know bits are bunched together. It saves memory but individual access can be	Access may be slower

		slower than normal bool arrays.	
T* pointer	Node<int>* p;	Pointers exist for C++-style low-level escape hatches. The default pointer value is null rather than uninitialized garbage.	O(1)
container defaults	List<int> L;	List, Arr, Set, Map, HashMap, Heap, Graph, and related containers default to empty containers when declared without an initializer.	O(1) or setup
int x;	int x;	Declares x and initializes it to the int default 0. CP++ avoids uninitialized local variables by design.	O(1)
int x = 5;	int x = 5;	Declares and initializes x normally. The declared type determines the storage type and allowed operations.	O(1)
int x = input();	int x = input();	Declares x and reads one int from input. The input call is target-typed by the declaration.	O(token length)
int a, b, c;	int a, b, c;	Multi-declares variables of the same type. Each receives the type default.	O(number of vars)
int a, b, c = 1, 2, 3;	int a,b,c = 1,2,3;	Multi-assignment. Values are assigned left-to-right to the variables declared in the group.	O(number of vars)
(int) f	int y = (int)avg;	Explicit downcast from a wider or fractional type to int. Downcasts are explicit. If float-to-int casting follows CP++ floor semantics, negative values floor rather than truncate.	O(1)
bigint b = x	bigint b = x;	Automatic upcast from int to bigint. Upcasts may be implicit because they do not lose information.	O(size of x)
type promotion	float y = x + 0.5;	Numeric promotion should preserve information when possible: int may promote to float or bigint; float combined with bigint promotes to bigfloat. Downcasts require explicit syntax.	Compile-time rule
sentinel values	print(M[missing]);	Missing/default values are false for bool, 0 for int/bigint/char, 0.0 for float/bigfloat, empty string for string, and empty container for containers.	Semantic rule

Usage notes

- The primitive type names are intentionally not exact C++ names. int means the integer type a competitive programmer normally wants, which is 64-bit signed.
- Default initialization is a contest-safety choice. It reduces uninitialized-value errors but does not remove the need to reason about whether a sentinel value is meaningful.
- The conversion rules try to avoid silent data loss. Upcasts may be automatic, but downcasts should be explicit so that precision loss is visible in the code.
- bigfloat exists mostly to make the type lattice coherent. It is not intended to become a commonly used contest type.

3. Operators, Arithmetic, Comparison, and Bit Operations

CP++ keeps C++-style operators where they help speed and familiarity, but changes selected semantics where CP/math ergonomics are better. The main deliberate change is floor division.

Feature / function	Example	Detailed explanation	Complexity / notes
+ - *	int z = a*b + c;	Standard numeric arithmetic. For strings and certain containers, + may mean concatenation. For lists, a + b returns a new concatenated list.	Numeric O(1); list concat O(n+m)
/	int q = -3 / 2;	Floor division for integer operands. Unlike C++, negative integer division should floor mathematically. For floating operands, normal floating division is used.	O(1)
%	int r = x % mod;	Modulo compatible with floor division. Ideally a == (a / b) * b + (a % b), with modulo behavior chosen consistently for negatives.	O(1)
+= -= *= /= %=	ans += cur;	Assignment versions exist for arithmetic operators. For containers, += and *= may have container-specific meanings such as extension or repetition.	Varies
^ & ~	mask = 1 << b;	Bitwise OR, XOR, AND, and NOT. These follow C++-style operator spelling. They are essential for bitmask DP and low-level contest tricks.	O(1) for int
x << k, x >> k	int y = x << 2;	Normal left and right shifts. For int, shifts operate on 64-bit values. For bigint, shifts may operate on arbitrary-precision integer representations.	O(1) or digit-size
x <~ k, x >~ k	int y = x <~ 3;	Circular left and right rotations. For int, rotation is over 64 bits. For bitvector, rotation is over len(B) bits. For bigint, rotation should be disallowed unless a width is specified.	O(1) int; O(len/word) bitvector
x[n]	if(mask[b]) print(b);	Integer bit indexing. Returns the nth bit of x as bool, where n = 0 is the least significant bit. This compresses the common ((x >> n) & 1) idiom.	O(1) for int
bits(x)	bitvector B = bits(mask);	Returns the bits of x, least-significant bit first. The target type determines whether the result is List<bool> or packed bitvector.	O(number of bits)
power2(x, <)	int p = power2(n, <);	Returns the largest power of two less than or equal to x. Requires x > 0. Useful for sizing, binary lifting, sparse tables, and tree internals.	O(1) or O(log bits)
power2(x, \)	int lb = power2(x, \);	Returns the largest power of two that divides x, also called lowbit. Equivalent to x & -x for positive integers. Requires x != 0.	O(1)
swap(a, b)	swap(L, R);	Swaps two variables of the same type. For containers, this should swap internal	O(1) for most containers

		buffers/handles when possible rather than copying all elements.	
== != < <= > >=	if(a[i] <= x) {...}	Comparison operators. Primitive types compare naturally; strings compare lexicographically; lists and pairs compare lexicographically; structs may compare by declaration order.	Varies
x in C	if(x in seen) print(x);	Membership operator. For List, checks whether x appears. For Set/Map/HashMap, checks key membership. For string, checks substring/character depending on operand type if defined.	List O(n); Set O(log n); Hash O(1)*
container truthiness	while(Q) process(Q.pop0);	A container used as a condition is true iff it is non-empty. This supports while(Q) loops for queues/heaps/stacks and if(L) checks.	O(1) if size stored
all(C)	if(all(good)) print("yes");	Returns true iff all boolean elements in C are true. Empty containers return true by vacuous truth.	O(n)
any(C)	if(any(hit)) print("found");	Returns true iff at least one boolean element in C is true. Empty containers return false.	O(n)

Usage notes

- The floor-division rule is mathematically cleaner than C++ truncation but creates a porting difference from C++. Any implementation must document negative division and modulo behavior carefully.
- Integer bit indexing is one of the clearest examples of CP++ compressing a common idiom. x[b] says exactly what ((x >> b) & 1) means.
- Rotate operators require width. Without a width, rotation is undefined as a concept. CP++ gives int a 64-bit width and bitvector its own length.
- swap must be cheap for containers. If swap copies all elements, it violates the expectation inherited from C++ container swap.

4. Control Flow, Ranges, Iteration, and Comprehensions

CP++ keeps C++-style braces for control flow while adopting Python-style iteration utilities where they remove boilerplate.

Feature / function	Example	Detailed explanation	Complexity / notes
if / else if / else	if(x<0){...} else {...}	Conditionals use C++ brace style: if(condition) { ... } else if(condition) { ... } else { ... }. Logical operators are &&, , and !.	Control flow
while(condition)	while(Q) {...}	Standard while loop. Common with queues/heaps because containers are truthy when non-empty.	Loop-dependent
rep(n)	rep(t) solve();	Repeats the body n times without exposing an index. This is for cases where the index is irrelevant, especially input loops.	O(n body)
for(int i = 0; i < n; i++)	for(int i=0;i<n;i++){...}	Classic C++ for loop remains supported for unusual step logic, performance-sensitive loops, or cases where range syntax is	Loop-dependent

		awkward.	
<code>for(int x in L)</code>	<code>for(int x in a) sum += x;</code>	Iterates over elements of a container. By default x is a value/copy unless a reference form is explicitly used.	O(n)
<code>range(b)</code>	<code>for(int i in range(n)) {...}</code>	Represents integers 0 through b-1. It is lazy in loops and materialized only when assigned to a concrete container.	O(1) lazy; O(n) materialized
<code>range(a, b)</code>	<code>for(int i in range(l,r)) {...}</code>	Half-open range from a toward b, excluding b. If a < b, step is +1; if a > b, step is -1. Thus <code>range(5,1)</code> yields 5,4,3,2.	O(1) lazy
<code>range(a, b, c)</code>	<code>for(int i in range(0,n,2)) {...}</code>	Explicit-step range using Python-style semantics. The step c must not be zero. If the explicit step moves away from b, the range is empty.	O(1) lazy
<code>for(int i, x in enumerate(L))</code>	<code>for(int i, x in enumerate(a)) {...}</code>	Iterates with both index and value. The first variable is the index, the second is the element. This replaces manual <code>range(len(L))</code> loops.	O(n)
<code>M.items()</code>	<code>for(int x,c in freq.items()) {...}</code>	Returns or lazily iterates key-value pairs from a map. In ordered Map, iteration is sorted by key. In HashMap, order is unspecified.	O(n)
<code>M.keys()</code>	<code>List<int> keys = M.keys();</code>	Returns a List of all keys. Ordered Map returns keys in sorted order; HashMap order is unspecified.	O(n)
<code>M.values()</code>	<code>List<int> vals = M.values();</code>	Returns a List of all values. For ordered Map, values follow key order. For HashMap, order is unspecified.	O(n)
<code>[expr for int x in C]</code>	<code>List<int> sq = [x*x for int x in a];</code>	List comprehension. Builds a List by evaluating expr for each x in C. The loop variable is statically typed.	O(n)
<code>{expr for int x in C}</code>	<code>Set<int> s = {x%m for int x in a};</code>	Set comprehension. Builds a Set, automatically removing duplicates and ordering by the set comparator.	O(n log n) ordered; O(n)* hash
<code>{k:v for int x in C}</code>	<code>Map<int,int> m = {x:x*x for int x in a};</code>	Map comprehension. Builds a Map from generated key-value pairs. Duplicate keys overwrite or error depending on final rule; overwrite is more Python-like.	O(n log n) ordered
if filter in comprehension	<code>List<int> e = [x for int x in a if x%2==0];</code>	Optional filter: <code>[expr for int x in C if condition]</code> . Only elements satisfying condition are emitted.	O(n)
<code>Gen<T> f(...) { yield x; }</code>	<code>Gen<int> odds(int n){ yield 1; }</code>	Defines a lazy generator function yielding values of type T. yield is only legal inside Gen<T> functions, making generator-ness explicit.	Lazy; per-yield cost
<code>yield x</code>	<code>yield cur;</code>	Produces one value from a generator and suspends until the next iteration. All yielded values must be compatible with the Gen<T> element type.	Per-yield

Usage notes

- CP++ intentionally keeps braces because nested contest code benefits from explicit block boundaries. The Python influence appears in iteration, not indentation.
- `range(a,b)` counting down when `a>b` is a deliberate CP++ difference from Python. It matches what many contest programmers mean when they write a descending range.
- Comprehensions should be used for construction, not for hiding complex algorithms. If the expression becomes unreadable, a normal loop is better.
- Generators are useful for custom lazy ranges such as grid neighbors. They should not be presented as the default fastest loop mechanism.

Enumerate and filtered comprehension

```
for(int i, x in enumerate(a)) {  
    if(x in bad) print(i, x);  
}  
List<int> squares = [i*i for int i in range(0,n) if i % 2 == 0];
```

5. Functions, Pass Modes, Pointers, and References

Functions are intentionally close to C++ so algorithmic helpers and recursive routines remain familiar. CP++ adds function-level pass modes to reduce accidental container copies.

Feature / function	Example	Detailed explanation	Complexity / notes
type name(params) { return x; }	int sq(int x){ return x*x; }	Standard function definition. The return type appears before the name. Parameters are statically typed.	Call-dependent
name(params) { ... }	show(int x){ print(x); }	Blank return type means void. This removes noise for helper functions that only print, mutate, or perform traversal.	Call-dependent
return x;	return ans;	Returns a value from a non-void function. In one-expression lambdas/comparators, CP++ may allow implicit return of the final expression.	O(1) plus value
List<int> L parameter	int f(List<int> L){...}	Containers are pass-by-value by default. int f(List<int> L) copies the entire list. This is honest but can be expensive.	O(n) copy
List<int>& L parameter	void f(List<int>& L){...}	Reference parameter. The function receives the original container; mutations are visible to the caller.	O(1) pass
const List<int>& L	int sum(const List<int>& L){...}	Read-only reference parameter, C++-style. Avoids copying while forbidding mutation.	O(1) pass
pass int f(List<int> L)	pass int f(List<int> L){...}	Function-level modifier: all container parameters are passed by mutable reference. Scalar parameters remain by value. This is CP++ shorthand for avoiding accidental vector copies.	O(1) container pass
const int f(List<int> L)	const int f(List<int> L){...}	Function-level modifier: all container parameters are passed by const reference. It has the same performance goal as pass but prevents mutation.	O(1) container pass
pointer defaults	Node<int>* p;	Pointers behave like basic C++ pointers, but default to null. This avoids uninitialized pointer garbage in short programs.	O(1)
inline lambda comparator	sort(a, bool(int x,int y){x>y;});	A function literal such as bool(List<int> a, List<int> b) { a[1] < b[1]; } can be passed to sort/Heap/Set. Comparator parameters should be lowered as const references when containers are involved.	Comparator-dependent
same-interface testing functions	Test[slow, fast].auto(1000);	Functions compared by Test[f1,f2] must have compatible parameter and return types. The testing framework uses this type information to generate cases.	Compile-time rule

Usage notes

- The pass and const function modifiers exist because accidental container copies are a classic performance bug. They make the common performance intention explicit at function level.

- CP++ keeps ordinary references and const references as escape hatches. Users who want C++-style precision can still write it directly.
- Blank return type means void, but only for definitions where the grammar is unambiguous. This is a convenience feature, not dynamic typing.
- Comparator lambdas should avoid copying large containers during sort. A backend should lower list-like comparator parameters to const references when safe.

6. Object Model: Pair, Struct, and Class

CP++ keeps records lightweight. Structs are methodless containers; classes are structs with constructors, methods, access modifiers, and static fields. This is contest OOP, not enterprise OOP.

Feature / function	Example	Detailed explanation	Complexity / notes
Pair<A,B>	Pair<int,string> p = 3,"cat";	A two-field product type. Pair<int,char> P = 2,'2'; creates a pair. Braced initialization should also be accepted.	O(1)
P[0], P[1]	print(P[0], P[1]);	Pairs are indexed by 0 and 1. Both fields are assignable. Out-of-range literal indices should be compile-time errors.	O(1)
struct A { ... };	struct Edge{int u;int v;int w;;}	Defines a methodless record type. Fields default to their type defaults. Structs are intended for named records like Edge, NodeState, or Interval.	Compile-time
A a;	Edge e;	Creates a struct with all fields set to defaults. This is useful for arrays of records without manual initialization.	O(number of fields)
A a = { ... };	Edge e = {u,v,w};	Positional struct initialization in declaration order. The values correspond to the fields as written in the struct body.	O(number of fields)
a.x	print(e.w);	Named field access. This is the preferred readable way to access struct fields.	O(1)
a[0]	print(e[0]);	Indexed field access. The first declared field is index 0. This is for generic sorting/comparison and compact CP code. It is not raw memory reinterpretation.	O(1)
struct comparison	sort(edges);	Structs may compare lexicographically by field declaration order, allowing sort(List<A>) when all fields are comparable.	O(number of fields)
class C { ... };	class DS{ List<int> p; };	Defines a class with public-by-default fields, constructors, methods, static fields, and optional private fields. CP++ classes are intended for DSU/Fenwick/Trie/custom helper objects, not enterprise architecture.	Compile-time
private int x;	private int secret;	Marks a field private. CP++ uses local access modifiers rather than C++ section labels, keeping the rule close to Java-style field declarations.	Compile-time
static int y;	static int count;	Defines class-level shared state. It behaves like Java/C++ static class data, not per-instance data.	O(1) access

<code>C(params) { ... }</code>	<code>DSU(int n){ parent=range(n); }</code>	Constructor. It initializes fields. Parameters and fields may use different names, or the language may support this. <code>x = x</code> style assignment.	Construction-dependent
method definitions	<code>int find(int x){...}</code>	Methods are functions inside a class. Blank return type means void. Methods can access instance fields and class static fields.	Call-dependent
<code>C c = C(params);</code>	<code>Solver s = Solver(n);</code>	Constructs a class object using assignment-style initialization, consistent with C++ container constructors.	Construction-dependent

Usage notes

- Pairs and structs are designed around records used in algorithms: edges, intervals, states, points, and DP nodes.
- Struct indexing is not memory-level indexing. It is syntactic access by declaration order. This preserves type safety while enabling compact compare and sort patterns.
- Classes are deliberately minimal. C++ OOP is for bundling state with helper methods, not modeling large application domains.
- Public-by-default fields are a contest convenience. If a class needs invariants, private fields and controlled methods should be used.

7. Common Container Protocols and Literals

Many C++ containers share method names and behaviors. This section defines the common protocol before the individual containers specialize it.

Feature / function	Example	Detailed explanation	Complexity / notes
<code>len(C)</code>	<code>int n = len(a);</code>	Returns number of elements/nodes/items. For Graph, <code>len(G)</code> means number of nodes. For Map, <code>len(M)</code> means number of keys.	Usually $O(1)$
<code>C.clear()</code>	<code>seen.clear();</code>	Removes all elements and returns the container to its empty state. For graph-like structures it should clear nodes, edges, and auxiliary visited state.	$O(n)$
<code>if(C)</code>	<code>if(L) print(L[-1]);</code>	Container truthiness. True if non-empty, false if empty. Common for <code>while(Q)</code> and <code>if(L)</code> .	$O(1)$
<code>C.add(x)</code>	<code>L.add(x);</code>	General insertion operation. Exact behavior depends on container: append for List, insert for Set, push for some structures, sorted insert for certain modes.	Container-dependent
<code>C.remove()</code>	<code>int x = S.pop();</code>	General removal operation. For List/Stack/Queue/Heap, it usually removes and returns the natural next/back/top/front element.	Container-dependent
<code>x in C</code>	<code>if(x in seen) print(x);</code>	Membership check. For maps, checks keys. For lists, scans values. For sets/hash sets, uses tree/hash membership.	Container-dependent
<code>min(C), max(C)</code>	<code>print(min(a), max(a));</code>	Returns minimum or maximum element/key. Lists require scanning. Ordered sets/maps can return in $O(1)$ or $O(\log n)$ depending on implementation.	List $O(n)$; ordered $O(1)/O(\log n)$
<code>sum(C)</code>	<code>int total = sum(a);</code>	Sums elements of numeric lists/sets/multisets. For Map, sums keys. Only valid for addable element/key types.	$O(n)$
<code>C + D</code>	<code>List<int> z = a + b;</code>	For sequence-like containers, returns a new concatenation. For heaps, returns a heap built by combining internal arrays and heapifying.	List $O(n+m)$; Heap $O(n+m)$
<code>C += D</code>	<code>a += b;</code>	Extends C by elements of D. For List this can be $O(\text{len}(D))$ amortized. For heaps it may rebuild/heapify to avoid repeated log factors.	Container-dependent
<code>C *= k</code>	<code>s *= 3;</code>	Repeats sequence-like containers k times. Supported for List, string, Multiset multiplicities, Queue/Stack/Deque underlying order, but not Set/Map.	$O(k*n)$
<code>[1,2,3]</code>	<code>List<int> a = [1,2,3];</code>	Sequence literal. Can initialize List, Stack, Queue, Deque, Heap, and other sequence-consuming containers.	$O(n)$
<code>{1,2,3}</code>	<code>Set<int> s = {1,2,2,3};</code>	Set-style literal. Can initialize Set/HashSet/Multiset depending on declared	$O(n \log n)$ or $O(n)*$

		target type. Ordered Set removes duplicates and sorts.	
{1:2, 3:4}	Map<int,int> m = {1:2,3:4};	Map literal. Initializes Map/HashMap with key-value pairs. Ordered Map stores keys in sorted order.	O(n log n) or O(n)*
range assigned to container	List<int> a = range(0,n);	List<int> R = range(1,5) materializes the lazy range into the target container. Works for sequence and set-like containers.	O(n)

Usage notes

- The shared container protocol is meant to reduce API switching cost. A programmer should not have to remember five different names for size, insertion, or clearing.
- The same method name may have different complexity on different containers. CP++ should never hide that in documentation.
- Container literals are target-typed. The same [1,2,3] can initialize a List, Queue, Stack, Deque, or Heap depending on the declared type.
- Truthiness is convenient, but it should only mean non-empty for containers. It should not become a general dynamic-language truthiness system.

8. List, Arr, String, and Bitvector

List is the default dynamic contiguous sequence, closer to C++ vector/Python list than to a linked list. Arr is fixed-size. String and bitvector share sequence-like behavior but have specialized storage/semantics.

Feature / function	Example	Detailed explanation	Complexity / notes
List<T> L;	List<int> L;	Declares an empty dynamic list of T. It is 0-indexed and optimized for random access and append.	O(1)
List^k<T>	List^3<int> dp;	Nested list shorthand. List^2<int> means List<List<int>>. List^4<int> is a four-dimensional dynamic list.	Type shorthand
L[i]	int last = L[-1];	Returns element at index i. Negative indexing is supported: L[-1] is the last element. Out-of-range access is a runtime error.	O(1)
L[a:b]	List<int> mid = L[1:-1];	Returns subarray/slice from index a inclusive to b exclusive, Python-style. Negative endpoints are allowed. The slice is a new list.	O(length of slice)
L.add(x)	L.add(5);	Appends x to the back of the list. This is the normal push-back operation. It may occasionally reallocate.	O(1)* amortized
L.add(x, i)	L.add(5, 2);	Inserts x at index i, shifting later elements right. Good for small lists; expensive for large lists.	O(n)
L.add(x, sorted)	L.add(5, sorted);	Assuming L is sorted, finds the correct insertion position and inserts x while preserving sorted order. Search is logarithmic but shifting dominates.	O(n)

L.remove()	int last = L.remove();	Removes and returns the last element. Empty removal is a runtime error.	O(1)
L.remove(i)	L.remove(3);	Removes and returns element at index i, shifting later elements left. This removes by index, not by value.	O(n)
L.index(x)	int i = L.index(7);	Returns the first index where L[i] == x, or -1 if absent. This is a linear search.	O(n)
L.index(x, sorted)	int i = L.index(7, sorted);	Binary-search version for sorted lists. Returns first occurrence if found, otherwise -1. Requires L to already be sorted.	O(log n)
L.lower(x)	int i = L.lower(x);	First index i such that L[i] >= x in a sorted list. Equivalent to lower_bound. Returns len(L) if no such index exists.	O(log n)
L.upper(x)	int j = L.upper(x);	First index i such that L[i] > x in a sorted list. Useful for counting values in ranges.	O(log n)
Arr<T> A[size]	Arr<int> a[n];	Declares a statically sized array. It is fixed-length and 0-indexed. It is intended when a fixed size is known at declaration.	O(size) init
string s;	string s = input();	Declares an empty string. Strings support len, indexing, slicing, comparison, input, and concatenation.	Varies
s[i], s[a:b]	char c = s[0]; string t=s[1:];	Character access and substring slicing. Negative indices behave like List negative indices.	O(1) index; O(k) slice
string multiplication	string pad = " " * 19;	A string may be repeated with *. For example " " * 19 creates 19 spaces, useful in formatted strings.	O(n*k)
bitvector B;	bitvector B = bits(mask);	Packed bit sequence. Supports indexing, assignment, len, add, count, and conversion from bits(x). Stores bits compactly rather than as full bool objects.	Access O(1) but slower constant
B[i] = true	B[3] = true;	Sets one packed bit. Implementation must update the relevant machine word and bit position.	O(1)
B.count(true)	int ones = B.count(true);	Counts number of set bits. Implementation may use popcount on machine words.	O(words)

Usage notes

- List is the default sequence because contiguous dynamic arrays dominate competitive programming. LinkedList exists, but it is specialized.
- Negative indexing and slicing are Python imports that are genuinely useful in algorithm code, especially for quick last-element access and substring/list manipulation.
- Sorted-list helpers do not make List a balanced tree. add(x, sorted) still shifts elements and remains linear overall.
- bitvector is for memory-sensitive boolean arrays and bitset-like tasks. It should not replace List<bool> when ordinary element speed matters more than memory.

Sequence and bitvector operations

```
List<int> a = input(n);
a.add(5);
```

```
List<int> middle = a[1:-1];  
bitvector B = bits(mask);  
if(mask[3]) print("third bit set");
```

9. Set, Map, Multiset, HashSet, and HashMap

CP++ separates ordered and hashed associative containers. Ordered containers are predictable and support min/max; hashed containers are faster on average but unordered.

Feature / function	Example	Detailed explanation	Complexity / notes
Set<T> S;	Set<int> S;	Ordered set of unique T values. Iteration is sorted by comparator. Supports add, delete, in, len, min, max, clear.	Most ops $O(\log n)$
Set<T> S(cmp);	Set<List<int>> S(compare(1));	Constructs an ordered set using custom comparator cmp. This replaces type-level comparator syntax. The type says what is stored; the constructor says how it is ordered.	Comparator-dependent
S.add(x)	S.add(x);	Inserts x if absent. Duplicate insertions have no effect.	$O(\log n)$
S.delete(x)	S.delete(x);	Removes x if present. If absent, either no-op or error depending on final strictness rule; no-op is more CP-friendly.	$O(\log n)$
Multiset<T> MS;	Multiset<int> MS;	Ordered multiset. Stores duplicates and supports count. Useful for sliding windows, greedy problems, and maintaining ordered bags.	$O(\log n)$ ops
MS.count(x)	print(MS.count(x));	Returns multiplicity of x in the multiset.	$O(\log n)$ or $O(\log n + \text{count})$
MS.decrement(x)	MS.decrement(x);	Removes one copy of x rather than all copies. This is the common safe version of C++ ms.erase(ms.find(x)).	$O(\log n)$
Map<K,V> M;	Map<int,int> M;	Ordered key-value map. Keys are sorted. Missing M[key] returns the value-type sentinel by design; M.at(key) is strict.	$O(\log n)$ key ops
Map<K,V> M(cmp);	Map<Node,int> M(compare("id"));	Constructs an ordered map with custom key comparator. The comparator applies to keys, not values.	$O(\log n)$ ops
M[key]	int c = freq[x];	Returns stored value if key exists; otherwise returns the value-type sentinel. Used for frequency maps and default adjacency lists. Assignment/mutation may insert the key.	$O(\log n)$
M.at(key)	int val = M.at(k);	Strict lookup. Returns stored value if present and errors if absent. Use when absence should not silently become a sentinel.	$O(\log n)$
M[key] = value	M[x] = i;	Assigns value to key, inserting the key if needed. This is the normal map update operation.	$O(\log n)$
M.delete(key)	M.delete(x);	Removes a key and its value from the map.	$O(\log n)$
key in M	if(x in M) print(M[x]);	Checks whether key exists. This is the only way to distinguish an absent key from a present key mapped to the sentinel value.	$O(\log n)$
M.keys()	List<int> keys = M.keys();	Returns sorted list of keys for Map, unspecified order for HashMap.	$O(n)$
M.values()	List<int> vals = M.values();	Returns values in key order for Map, unspecified order for HashMap.	$O(n)$

M.items()	for(int x,c in freq.items()) {...}	Returns or iterates Pair<K,V> key-value pairs. In loops, for(K k, V v in M.items()) is supported.	O(n)
HashSet<T> HS;	HashSet<int> HS;	Hashed unique set. Faster average membership/insertion but unordered and vulnerable to adversarial hashing unless implementation protects it.	O(1)* expected
HashMap<K,V> HM;	HashMap<int,int> HM;	Hashed key-value map. Same sentinel and at semantics as Map, but average O(1) access.	O(1)* expected
Counter(V)	Map<int,int> f = Counter(a);	Builds a frequency Map from a container V. The output type may be Map<T,int> or HashMap<T,int> depending on target type.	O(n log n) Map; O(n)* HashMap
max(M, x)	int k = max(M, x);	Returns the maximum key in ordered M that is less than x, based on the earlier design. This is a predecessor helper for ordered containers.	O(log n)

Usage notes

- Map and HashMap share surface semantics but not performance or order. Ordered Map is predictable; HashMap is faster on average.
- Sentinel map access is excellent for frequencies and adjacency lists, but strict at() must be used when missing keys are logically errors.
- Counter is not magic. It just formalizes the frequency-map loop that contest programmers write constantly.
- Custom comparators belong in constructors so the type remains the stored type and the construction describes ordering behavior.

Counter and map item iteration

```
Map<int,int> freq = Counter(a);
for(int x, cnt in freq.items()) {
    if(cnt > 1) print(x, cnt);
}
```

10. Heap, Stack, Queue, and Deque

These containers expose common CP access patterns while avoiding C++ priority_queue/front/pop ceremony. Pop operations return the removed value.

Feature / function	Example	Detailed explanation	Complexity / notes
Heap<T> H;	Heap<int> H;	Default max-heap. The top element is the largest according to the comparator.	O(1) construct
Heap<T> H(min);	Heap<int> H(min);	Min-heap constructor. The heap stores T but orders so the smallest element is top. This replaces Heap<T,min> type-level syntax.	O(1) construct
Heap<T> H(cmp);	Heap<Node> H(compare("dist"));	Heap with custom comparator or key comparator such as compare(1). The comparator controls priority order.	Comparator-dependent
Heap<T> H = [values]	Heap<int> H = [3,1,4];	Constructs a heap from a sequence literal by heapifying, not repeated pushing.	O(n)

Heap<T> H = Heap(V)	Heap<int> H = Heap(a);	Heapifies an existing list/iterable V into a heap.	O(n)
H.top()	int x = H.top();	Returns top element without removal. Empty heap top is a runtime error.	O(1)
H.pop()	int x = H.pop();	Removes and returns top element. This fixes C++ priority_queue.pop() not returning the value.	O(log n)
H.push(x)	H.push(x);	Pushes x into the heap.	O(log n)
H + G	Heap<int> Z = H + G;	Merges two heaps by combining internal arrays and heapifying. This should be O(n+m), better than pushing all elements one-by-one.	O(n+m)
Stack<T> S = {values}	Stack<int> S = {1,2,3};	Stack initialized from a sequence. The back/end is the top. If values are {1,2,3}, S.pop() returns 3 first.	O(n)
S.push(x)	S.push(x);	Pushes x onto the top of the stack.	O(1)*
S.pop()	int x = S.pop();	Removes and returns the top element. Empty pop is runtime error.	O(1)
Queue<T> Q = {values}	Queue<int> Q = {1,2,3};	Queue initialized from a sequence. The front is the first element. If values are {1,2,3}, Q.pop() returns 1 first.	O(n)
Q.push(x)	Q.push(x);	Enqueues x at the back.	O(1)*
Q.pop()	int u = Q.pop();	Dequeues and returns the front element.	O(1)
Deque<T> D;	Deque<int> D;	Double-ended queue. Supports efficient push/pop at both ends.	O(1) construct
D.push_back(x)	D.push_back(x);	Adds x to the back.	O(1)*
D.push_front(x)	D.push_front(x);	Adds x to the front.	O(1)*
D.pop_back()	int x = D.pop_back();	Removes and returns the back element.	O(1)
D.pop_front()	int x = D.pop_front();	Removes and returns the front element.	O(1)

Usage notes

- The heap API is designed to fix the C++ priority_queue pain point. pop returns the removed value because that is what programmers almost always want.
- Stack and Queue exist even though List can simulate many stack use cases. Their value is semantic clarity and safe O(1) front/back behavior.
- Deque should be the default choice when both ends matter. LinkedList should not be used merely to get front insertion or deletion.
- Heap merging should use heapify, not repeated push, when both heaps can expose their backing arrays.

11. LinkedList and Node Handles

LinkedList exists only because some problems need node identity, circular traversal, or value-to-node lookup. List<T> remains the default sequence. LinkedList is specialized and should not be used as a normal vector replacement.

Feature / function	Example	Detailed explanation	Complexity / notes
LinkedList<T> L;	LinkedList<int> L;	Default doubly linked list. Supports node handles, add/remove, indexing by traversal, and node movement.	O(1) construct
LinkedList<T> L = input(n)	LinkedList<int> L = input(n);	Reads n values into a linked list. Input modes mirror List input, but storage is node-based.	O(n)
LinkedList<T, circ> L;	LinkedList<int,circ> L;	Circular doubly linked list. Node next/prev movement wraps around the ends. Useful for Josephus and cyclic simulations.	Node movement wraps
LinkedList<T, sing> L;	LinkedList<int,sing> L;	Singly linked list. Stores next pointers only. prev operations are not available.	Less memory; no prev
LinkedList<T, HashMap> L;	LinkedList<int,HashMap> L;	Doubly linked list with value-to-node hash map. Values must be unique. add appends to the end and updates the hash map.	find O(1)*, add O(1)*
LinkedList<T, Map> L;	LinkedList<int,Map> L;	Doubly linked list with ordered value-to-node map. Values must be unique. The list is sorted by value; add inserts in sorted position using the map.	find/add O(log n)
LinkedList<T, Map, sing>	LinkedList<int,Map,sing> L;	Modifiers may be combined. This is a singly linked, map-backed, sorted unique linked list if the implementation supports that combination.	Mode-dependent
len(L)	int n = len(L);	Returns number of nodes. The list should store size so len is not a traversal.	O(1)
L.add(x)	L.add(5);	For normal and HashMap-backed lists, appends x at the back. For Map-backed lists, inserts x in sorted order. Duplicate values are disallowed in map-backed modes.	Normal O(1); Hash O(1)*; Map O(log n)
L.remove()	int last = L.remove();	Removes and returns the back value. Empty removal is runtime error.	O(1)
L.remove(N)	L.remove(node);	Removes the given node handle from the list. This is the main reason linked lists exist: deletion by known node is O(1).	O(1)
L[i]	int last = L[-1];	Returns value at index i by traversal. Negative indices may be supported but remain traversal-based. This is not random access.	Doubly O(min(i,n-i)); singly O(i)
L.node(i)	Node<int> N = L.node(3);	Returns the node at index i for ordinary linked lists. This is explicit node access and replaces target-type magic.	Doubly O(min(i,n-i)); singly O(i)
L.find(x)	Node<int> N = L.find(x);	For map-backed or hashmap-backed linked lists, returns the node whose value is x. For ordinary linked lists it may exist as linear search, but map modes are the intended use.	Hash O(1)*; Map O(log n); normal O(n)

N.next()	N = N.next();	Returns/moves to the next node by one step. On circular lists, wraps. On non-circular lists, stepping past the end yields null or runtime error depending on final rule.	O(1)
N.next(k)	N = N.next(k);	Moves k steps forward from node N. For circular lists, wraps repeatedly. For non-circular lists, cannot move past end safely.	O(k)
N.prev()	N = N.prev();	Returns/moves to the previous node in doubly linked lists. Not available in singly linked lists.	O(1)
N.prev(k)	N = N.prev(k);	Moves k steps backward in doubly linked lists. Circular lists wrap.	O(k)
N.val	print(N.val);	Direct value access. Allowed for normal linked lists. In map-backed lists, direct mutation would break the backing map and should be disallowed.	O(1)
N.update(y)	N.update(y);	Safe update of a node value. For mapped lists it repairs the map/hashmap and may reposition the node if the Map-backed list is sorted. Discouraged unless needed.	Normal O(1); Hash O(1)*; Map O(log n) or more
L.update(i, y)	L.update(5, y);	For ordinary linked lists, updates index i to y. For map-backed linked lists, the first argument is a value to find, not an index. This mode-dependent meaning must be documented loudly.	Mode-dependent

Usage notes

- LinkedList earns its place only through node identity: removing a known node, moving through next/prev, circular simulations, and value-to-node maps.
- HashMap-backed LinkedList preserves insertion order and gives expected O(1) value lookup. Map-backed LinkedList is sorted by value and gives ordered lookup.
- Direct node value mutation is safe only when no backing map depends on values. update exists because mapped lists must repair indexes.
- The node(i) versus find(x) split is important. Index-to-node and value-to-node should not be the same syntax.

Move a unique value to the back

```
LinkedList<int, HashMap> L = input(n);
Node<int> z = L.find(x);
L.remove(z);
L.add(x);
```

12. Graph and WeightedGraph

Graph helpers remove boilerplate without pretending to solve BFS/DFS for you. CP++ avoids canned BFS/DFS because real contest traversals often include custom state.

Feature / function	Example	Detailed explanation	Complexity / notes
Graph<T> G;	Graph<int> G;	Directed graph whose node labels have type	Construct O(1)

		T. Nodes may be arbitrary values, not necessarily 0..n-1.	
WeightedGraph<T,W> W;	WeightedGraph<int,int> W;	Directed weighted graph. Node labels have type T; edge weights have type W.	Construct O(1)
G.add(a,b)	G.add(u, v);	Adds directed edge a -> b. Nodes may be automatically added if absent.	Adjacency-dependent
W.add(a,b,c)	W.add(u, v, w);	Adds directed weighted edge a -> b with weight c.	Adjacency-dependent
G.add(a,b,*)	G.add(u, v, *);	Adds both a -> b and b -> a. The star marker means mirror/bidirectional edge. For weighted graphs use W.add(a,b,c,*).	Two edge inserts
G.addNode(a)	G.addNode(u);	Adds isolated node a if not already present.	Map/hash dependent
G.removeNode(a)	G.removeNode(u);	Removes node a and all incident edges. This can be expensive unless reverse adjacency is stored.	Potentially O(V+E)
G.remove(a,b)	G.remove(u, v);	Removes directed edge a -> b.	Adjacency-dependent
G.remove(a,b,*)	G.remove(u, v, *);	Removes both directions between a and b.	Two removals
G.degree(u)	int d = G.degree(u);	Returns outgoing degree of u. In undirected graphs represented by mirrored edges, this is the normal degree.	O(1) if stored
G.adj(u)	for(int v in G.adj(u)) {...}	Returns outgoing neighbors of u as List<T>. This is the core traversal helper.	O(outdegree)
W.adj(u)	for(Pair<int,int> e in W.adj(u)) {...}	Returns outgoing weighted neighbors as List<Pair<T,W>>, where pair[0] is neighbor and pair[1] is weight.	O(outdegree)
G.vis(u)	G.vis(u);	Marks u visited in the graph-local visited set. Visited state is a convenience helper, not a replacement for custom traversal state.	O(1)* or O(log n)
G.seen(u)	if(!G.seen(v)) dfs(v);	Checks whether u has been marked visited. Unknown nodes should normally return false.	O(1)* or O(log n)
G.resetVis()	G.resetVis();	Clears the visited set for the graph. Useful between DFS/BFS passes.	O(number visited)
no built-in BFS/DFS	// write custom BFS/DFS	CP++ deliberately avoids canned BFS/DFS because contest BFS state often includes parity, masks, coupons, previous edge colors, levels, parents, or other custom state.	Design rule

Usage notes

- Graph is intentionally not a full graph-algorithm library. It reduces adjacency and visited boilerplate while leaving BFS/DFS logic under programmer control.
- Directed edges are the default because undirected graphs can be represented by mirrored directed edges, while directed information cannot be recovered if the default is undirected.
- Weighted adjacency returns pairs so traversal logic stays explicit: e[0] is neighbor and e[1] is weight.
- Graph-local visited state is convenient but not enough for algorithms requiring colors, timestamps, parents, distances, or multidimensional state.

Custom DFS with built-in visited helpers


```
Graph<int> g;  
g.add(1, 2, *);  
  
dfs(int u) {  
    g.vis(u);  
    for(int v in g.adj(u)) {  
        if(!g.seen(v)) dfs(v);  
    }  
}
```

13. Sorting, Comparison, and Selection Helpers

Sorting by fields, indices, or custom comparators is a major C++ target. The design turns common comparator patterns into one-liners.

Feature / function	Example	Detailed explanation	Complexity / notes
<code>sort(L)</code>	<code>sort(a);</code>	Sorts a list/container normally using the natural comparator. Primitive types, strings, lists, pairs, and structs are comparable if their contents are comparable.	$O(n \log n)$
<code>sort(L, reverse)</code>	<code>sort(a, reverse);</code>	Sorts in descending natural order. <code>reverse</code> is a sorting modifier and should appear as the final sort argument.	$O(n \log n)$
<code>sort(L, foo)</code>	<code>sort(a, cmp);</code>	Sorts using external comparator <code>foo</code> . <code>foo(a,b)</code> should return true if <code>a</code> comes before <code>b</code> .	$O(n \log n * \text{cmp})$
<code>sort(L, foo, reverse)</code>	<code>sort(a, cmp, reverse);</code>	Sorts using comparator <code>foo</code> , then reverses comparator order. This handles descending custom sorts.	$O(n \log n * \text{cmp})$
<code>compare(i)</code>	<code>sort(rows, compare(1));</code>	Comparator that compares indexable objects by element <code>i</code> . Negative indices are allowed; <code>compare(-1)</code> compares the last element.	Comparator $O(1)$ or field access
<code>compare(i,j,k)</code>	<code>sort(rows, compare(0,1,-1));</code>	Multi-key comparator. Compares by <code>i</code> , then <code>j</code> , then <code>k</code> lexicographically. Useful for intervals, points, triples, and rows.	$O(\text{number of keys})$
<code>compare("field")</code>	<code>sort(edges, compare("w"));</code>	Comparator for structs/classes by named field. The field name should be checked against the type if possible to avoid typo bugs.	$O(\text{field access})$
<code>compare("a","b")</code>	<code>sort(items, compare("a","b"));</code>	Multi-field comparator for records. Compares <code>.a</code> first, then <code>.b</code> , etc.	$O(\text{number of fields})$
<code>sort(A, compare(1), reverse)</code>	<code>sort(rows, compare(1), reverse);</code>	Sorts by the second element descending. This replaces common lambdas like <code>a[1] > b[1]</code> .	$O(n \log n)$
<code>reverse(C)</code>	<code>reverse(a);</code>	Reverses sequence-like containers in place or returns reversed container depending on final rule. Used for List/string/deque-like containers, not Set/Map.	$O(n)$
<code>L.lower(x)</code>	<code>int i = L.lower(x);</code>	First sorted-list index with value $\geq x$. Used for insertion positions and count queries.	$O(\log n)$
<code>L.upper(x)</code>	<code>int j = L.upper(x);</code>	First sorted-list index with value $> x$.	$O(\log n)$
<code>first(lo, hi, fun)</code>	<code>int x = first(0,n,ok);</code>	Binary-search helper. Returns smallest <code>x</code> in inclusive <code>[lo,hi]</code> satisfying monotone predicate <code>fun</code> . If none exists, returns <code>hi+1</code> .	$O(\log \text{range} * \text{fun})$
<code>last(lo, hi, fun)</code>	<code>int x = last(0,n,ok);</code>	Binary-search helper. Returns largest <code>x</code> in inclusive <code>[lo,hi]</code> satisfying monotone predicate <code>fun</code> . If none exists, returns <code>lo-1</code> .	$O(\log \text{range} * \text{fun})$
<code>prefix(L, op, init)</code>	<code>List<int> p = prefix(a, max, -INF);</code>	Builds prefix values using operation <code>op</code> and initial identity <code>init</code> . <code>P[0] = init</code> , <code>P[i+1] = op(P[i], L[i])</code> .	$O(n)$
<code>prefix(L, sum)</code>	<code>List<int> p = prefix(a, sum);</code>	Common shorthand for prefix sums. Returns length <code>n+1</code> , starting with 0.	$O(n)$

Usage notes

- `compare(...)` is one of CP++'s strongest syntax-compression tools. It removes most custom comparator lambdas without preventing custom comparators when needed.
- `reverse` is a sort modifier rather than part of `compare` so the key selection and direction remain separate concepts.
- `first` and `last` encode monotone binary search. They should not be used when the predicate is not monotone.
- `prefix` generalizes prefix sums, but custom operations need valid identities and associativity if range reasoning depends on them.

14. Math, Number Theory, Randomness, FFT, and Constants

CP++ includes the helper functions serious contestants normally copy into templates. These helpers are not proof substitutes; they are standard algorithmic tools.

Feature / function	Example	Detailed explanation	Complexity / notes
INF	<code>int best = INF;</code>	Default large positive integer constant, roughly $4e18$. Used for distances and min segment tree identities.	Constant
LLMIN / LLMAX	<code>int low = LLMIN;</code>	True 64-bit signed limits, matching C++ long long min/max. Useful when exact machine bounds matter.	Constant
MOD	<code>int m = MOD;</code>	Default common prime modulus 1000000007.	Constant
MOD1	<code>int mod = MOD1;</code>	NTT-friendly modulus 998244353. Common in AtCoder and convolution problems.	Constant
<code>abs(x)</code>	<code>int d = abs(a-b);</code>	Absolute value for numeric types.	$O(1)$ or size
<code>ceil(x)</code>	<code>int c = ceil(avg);</code>	Ceiling conversion for floating values. Usually returns an int-like result.	$O(1)$
<code>round(x)</code>	<code>int r = round(x);</code>	Rounds to nearest integer or declared target format. Method-style <code>x.round(k)</code> may round display to <code>k</code> places.	$O(1)$
<code>clamp(x,l,r)</code>	<code>x = clamp(x, 0, n-1);</code>	Restricts <code>x</code> to the inclusive range <code>[l,r]</code> . Requires <code>l <= r</code> .	$O(1)$
<code>pow(a,b)</code>	<code>int p = pow(2, k);</code>	Fast exponentiation for integer-like types when <code>b</code> is nonnegative. Float powers may use floating exponentiation.	$O(\log b)$
<code>pow(a,b,mod)</code>	<code>int p = pow(a, b, MOD);</code>	Modular exponentiation: $a^b \bmod \text{mod}$, computed without forming a^b first.	$O(\log b)$
<code>pow(a,-1,mod)</code>	<code>int inv = pow(a, -1, MOD);</code>	Modular inverse of <code>a</code> modulo <code>mod</code> . Requires $\text{gcd}(a,\text{mod})=1$; otherwise runtime error.	$O(\log \text{mod})$
<code>div(a,b)</code>	<code>int q = div(a,b);</code>	Equivalent to <code>a / b</code> under CP++ floor division semantics.	$O(1)$
<code>div(a,b,mod)</code>	<code>int x = div(a,b,MOD);</code>	Modular division: $a * \text{inverse}(b) \bmod \text{mod}$. Requires <code>b</code> invertible modulo <code>mod</code> .	$O(\log \text{mod})$
<code>gcd(a,b)</code>	<code>int g = gcd(a,b);</code>	Greatest common divisor of integer-like values. Returns nonnegative result.	$O(\log \min(a,b))$
<code>lcm(a,b)</code>	<code>int l = lcm(a,b);</code>	Least common multiple. <code>lcm(0,x)</code> should be 0.	$O(\log \min(a,b))$

choose(n,k)	bigint c = choose(n,k);	Exact binomial coefficient $n \choose k$. Should return bigint by default to avoid overflow. Returns 0 for $k < 0$ or $k > n$.	Depends on n,k
choose(n,k,mod)	int c = choose(n,k,MOD);	$n \choose k$ modulo mod. Implementation may use factorial precomputation/caching when appropriate.	Depends on method
isPrime(n)	if(isPrime(n)) print(n);	Primality test. For int, deterministic Miller-Rabin with 64-bit-safe bases. For bigint, likely probable-prime unless otherwise specified.	Polylog expected
isPrime(n,*)	bool cert = isPrime(n,*);	Deterministic theoretical primality, intended as AKS-style proof mode. Very slow and mostly symbolic/educational.	Polynomial but impractical
sieve(n)	List<int> p = sieve(n);	Returns List<int> of all primes less than n.	$O(n \log \log n)$
factor(n)	List<int> d = factor(n);	Returns sorted positive divisors of abs(n). n=0 is invalid because zero has infinitely many divisors.	$O(\sqrt{n})$ or better
prime_factors(n)	Map<int,int> pf = prime_factors(n);	Returns Map<int,int> from prime factor to exponent for abs(n). Keys are sorted in ordered Map.	Depends on factoring
rand()	float r = rand();	Random float in [0,1). Not cryptographic.	$O(1)$
rand(a,b)	int x = rand(1,n);	If a and b are integer-like, returns random int in [a,b]. If float-like, returns random float in [a,b).	$O(1)$
rand(V)	int x = rand(a);	Returns uniformly random element of a non-empty container. For Map, should return a random key unless items() is used. Empty container is runtime error.	List $O(1)$; set/map may be $O(n)$
seed(x)	seed(1337);	Sets random seed for reproducible random/stress tests. Important for Test.auto and randomized algorithms.	$O(1)$
FFT(A,B)	List<int> C = FFT(A,B);	Returns convolution/product of coefficient arrays A and B using FFT internally. This is CP slang: the function name is FFT but the result is convolution.	$O(n \log n)$
FFT(A,B,mod)	List<int> C = FFT(A,B,MOD1);	Returns convolution modulo mod. For MOD1 = 998244353, implementation should use NTT. For arbitrary mod, CRT or fallback may be needed.	$O(n \log n)$

Usage notes

- The math library is a CP template library made standard. It is not trying to become a symbolic algebra system.
- FFT(A,B) is intentionally CP slang for convolution. The manual must state this clearly because mathematically FFT is the transform, not the product.
- Randomness is for stress tests, hashing, randomized algorithms, and pivots. It is not cryptographic unless a separate secure API is added.
- Number-theory helpers should prefer mathematically safe behavior, such as errors for nonexistent modular inverses rather than silent sentinels.

15. Standard Algorithmic Data Structures

These are built into C++ because they are not exotic in contests; they are repeated template material. The language should still document their mathematical contracts clearly.

Feature / function	Example	Detailed explanation	Complexity / notes
<code>SegTree<T> S = SegTree(n, op, id)</code>	<code>SegTree<int> S = SegTree(n, sum, 0);</code>	Creates a point-update range-query segment tree of size n , with all values initialized to identity id . op must be associative, and id must be the identity.	Build $O(n)$
<code>SegTree<T> S = SegTree(V, op, id)</code>	<code>SegTree<int> S = SegTree(a, min, INF);</code>	Builds a segment tree from List V in linear time if possible.	$O(n)$
<code>S[i]</code>	<code>int cur = S[i];</code>	Gets current value at index i . May be $O(1)$ if the tree stores the base array, or $O(\log n)$ if implemented via <code>query(i, i+1)</code> .	$O(1)$ or $O(\log n)$
<code>S[i] = x</code>	<code>S[i] = x;</code>	Point update: sets index i to x and recomputes affected segment tree nodes.	$O(\log n)$
<code>S.query(l, r)</code>	<code>print(S.query(l, r));</code>	Range query over half-open interval $[l, r)$. Returns op over the elements in that range. Empty range returns identity.	$O(\log n)$
not lazy SegTree	// no range add/assign	This SegTree does not support range update or lazy propagation. Range add/range assign requires a separate LazySegTree design.	Design rule
<code>Fenwick<T> F = Fenwick(n)</code>	<code>Fenwick<int> F = Fenwick(n);</code>	Creates additive Fenwick tree of size n , initialized to zero. T must support $+$, $-$, and zero.	$O(n)$ init
<code>Fenwick<T> F = Fenwick(V)</code>	<code>Fenwick<int> F = Fenwick(a);</code>	Builds Fenwick tree from a list. May be $O(n)$ with optimized construction or $O(n \log n)$ in a simple implementation.	$O(n)$ preferred
<code>F.add(i, x)</code>	<code>F.add(i, delta);</code>	Adds x to index i . This is the native Fenwick operation.	$O(\log n)$
<code>F[i]</code>	<code>int cur = F[i];</code>	Gets current value at i . Usually implemented as <code>query(i, i+1)</code> .	$O(\log n)$
<code>F[i] = x</code>	<code>F[i] = x;</code>	Sets index i to x by computing $delta = x - F[i]$, then <code>add(i, delta)</code> .	$O(\log n)$
<code>F.query(l, r)</code>	<code>print(F.query(l, r));</code>	Returns sum over half-open interval $[l, r)$.	$O(\log n)$
<code>DSU<int> D = DSU(n)</code>	<code>DSU<int> D = DSU(n);</code>	Creates disjoint set union over nodes $0..n-1$. Array-backed and fast.	$O(n)$ init
<code>DSU<T> D = DSU(V)</code>	<code>DSU<string> D = DSU(names);</code>	Creates DSU over unique values from V . Duplicates collapse. For non-int labels, uses map/hash internally.	$O(n \log n)$ or $O(n)^*$
<code>D.find(x)</code>	<code>int r = D.find(x);</code>	Returns representative of component containing x . Missing x should be an error, not silent insertion.	Amortized inverse Ackermann
<code>D.merge(x, y)</code>	<code>if(D.merge(u, v)) ans += w;</code>	Unions components. Returns true if a merge actually occurred, false if already connected. Perfect for Kruskal.	Amortized inverse Ackermann
<code>D.same(x, y)</code>	<code>if(D.same(a, b)) print("yes");</code>	Returns whether x and y are already in the same component.	Amortized inverse Ackermann
<code>len(D)</code>	<code>print(len(D));</code>	Returns number of nodes in the DSU, not number of components.	$O(1)$

D.comps()	print(D.comps());	Returns current number of connected components.	O(1)
D.compsize(x)	print(D.compsize(x));	Returns size of the component containing x.	Amortized inverse Ackermann

Usage notes

- SegTree is generic over the node type, not just int. This matters for max-count nodes, DP states, matrix products, and custom monoids.
- Fenwick is intentionally narrower than SegTree. It is add-only, lighter, and faster for the common prefix-sum/update pattern.
- DSU<int> should be array-backed and fast. DSU<string> or other arbitrary labels should be viewed as convenience with map/hash overhead.
- These data structures should be documented as standard library tools, not language syntax. Users still need to understand when their use is valid.

Standard data structure usage

```
SegTree<int> S = SegTree(a, sum, 0);
S[i] = x;
print(S.query(l, r));
```

```
DSU<int> D = DSU(n);
if(D.merge(u, v)) ans += w;
```

16. Testing and Stress-Testing Framework

CP++ testing is designed around counterexample discovery. It supports ordinary assertions, differential testing, and brute-vs-fast stress tests generated from type information.

Feature / function	Example	Detailed explanation	Complexity / notes
Test t(f1, f2)	Test t(slow, fast);	Creates a differential tester comparing two functions with the same interface. The intended use is slow brute force versus fast optimized solution.	Setup O(1)
t.test({params})	t.test({{1,2,3}});	Runs both functions on explicit params and passes iff outputs are equal. Prints the mismatch if not equal.	Cost f1+f2
Test[f1,f2].test(params)	Test[slow,fast].test(a);	Shorthand for one differential test without naming a Test object.	Cost f1+f2
Test[f(params)].equals(val)	Test[gcd(12,18)].equals(6);	Manual assertion: evaluates f(params) and checks equality with val.	Cost f
Test[expr].notNull()	Test[p].notNull();	Passes iff expr is not null. Useful for pointer/node/optional-like APIs.	Cost expr
Test[expr].isNull()	Test[p].isNull();	Passes iff expr is null.	Cost expr
Test[expr].error(E)	Test[M.at(x)].error(KeyError);	Passes iff evaluating expr raises error type E. A no-argument error() form may accept any error.	Cost until error
Test[expr].greaterThan(x)	Test(ans).greaterThan(0);	Passes iff expr > x.	Cost expr
Test[expr].lessThan(x)	Test(dist).lessThan(INF);	Passes iff expr < x.	Cost expr
Test[f1,f2].auto()	Test[brute,fast].auto();	Generates a default set of small cases from	Generated case count

		function parameter types, compares f1 and f2, and prints the first failing case. This is not proof; it is stress testing.	
Test[f1,f2].auto(n)	Test[brute,fast].auto(10000);	Runs n generated tests. This is the standard brute-vs-fast stress testing form.	$n * (f1+f2)$
Test[f1,f2].auto(limit)	Test[brute,fast].auto(valid);	Runs generated tests satisfying predicate limit. The predicate has the same parameters as f1/f2 and filters valid inputs.	Depends on generation
Test[f1,f2].auto(n, limit)	Test[brute,fast].auto(10000, valid);	Runs n valid generated tests satisfying limit. If generation rejects too many candidates, implementation should avoid infinite loops or report failure to generate.	Depends on generation
type-based generation	Test[slow,fast].auto(100);	auto() should generate small values: bools, small ints, short strings, short Lists, Pairs, structs, sets/maps, and edge cases such as empty lists and repeated values.	Design rule
failure report	// prints first failing case	On failure, Test should print the case number, input parameters, f1 output, and f2 output. Reproducibility should include seed control.	O(size of report)
spiritual TC#2	// small hidden-case generator	auto() does not literally fetch a judge test. It recreates the spirit of hidden sample-breakers: small, dumb, adversarial cases that expose a wrong greedy or edge-case bug.	Design principle

Usage notes

- The Test system is not mainly enterprise unit testing. It is built around algorithm validation and counterexample discovery.
- auto() should be reproducible under seed control. A failing random case is only useful if the programmer can rerun it.
- limit predicates are essential because real problem inputs have constraints: even length, connected graph, valid edges, sorted inputs, and so on.
- Stress testing does not replace proof. It helps find bugs and develop confidence before proof or submission.

Differential stress testing

```
int brute(List<int> a) { /* slow but obvious */ }
int fast(List<int> a) { /* optimized */ }
```

```
Test[brute, fast].auto(10000, valid);
```


17. Example Idioms and Short Programs

This section shows how CP++ features compose. The examples are intentionally short but should be precise enough to guide implementation.

Feature / function	Example	Detailed explanation	Complexity / notes
Two Sum pattern	<code>if(t-x in seen) print(seen[t-x],i);</code>	Use <code>Map<int,int> seen</code> ; iterate with <code>enumerate</code> ; check <code>t-x</code> in <code>seen</code> ; then assign <code>seen[x] = i</code> . Sentinel map access plus membership gives clear logic.	$O(n \log n)$ Map; $O(n)$ * HashMap
Kruskal pattern	<code>sort(edges, compare("w"));</code>	Sort edges by <code>compare("w")</code> , create DSU over nodes, and add edge weight when merge returns true. This is the canonical DSU showcase.	$O(m \log m)$
Dynamic range sum	<code>F[i]=x; print(F.query(l,r));</code>	<code>Fenwick<int> F = Fenwick(a); F[i] = x; print(F.query(l,r));</code> This encodes point set and range sum directly.	$O(\log n)$ per query/update
Prefix sums	<code>List<int> P = prefix(a,sum);</code>	<code>List<int> P = prefix(a, sum);</code> range sum <code>a[l:r]</code> is <code>P[r] - P[l]</code> .	Build $O(n)$, query $O(1)$
Frequency map	<code>Map<int,int> f = Counter(a);</code>	<code>Map<int,int> freq = Counter(a);</code> for <code>(int x, cnt in freq.items()) { ... }</code>	$O(n \log n)$
Sort rows by second column	<code>sort(rows, compare(1));</code>	<code>sort(rows, compare(1));</code> descending uses <code>sort(rows, compare(1), reverse)</code> . Multi-key uses <code>compare(0,1,-1)</code> .	$O(n \log n)$
Interactive output	<code>print(query, flush);</code>	<code>print(query, flush);</code> <code>int response = input();</code> Normal print remains buffered so non-interactive code is fast.	I/O dependent
Brute-vs-fast validation	<code>Test[brute,fast].auto(1000);</code>	Write <code>slow()</code> and <code>fast()</code> , then run <code>Test[slow, fast].auto(10000, valid);</code> before submitting.	Testing dependent
Circular simulation	<code>cur = cur.next(k);</code>	<code>LinkedList<int,circ,HashMap> L = input(n);</code> <code>Node<int> cur = L.find(start);</code> <code>cur = cur.next(k);</code> <code>L.remove(cur);</code>	Depends on k and operations
Bitmask DP	<code>if(mask[b]) dp[mask]++;</code>	Use <code>x[b]</code> instead of <code>((x>>b)&1)</code> , <code>power2(mask, \)</code> for lowbit, and bitvector when memory matters.	Bit-dependent

Usage notes

- The idioms show the core CP++ promise: the code should resemble the algorithmic plan stated in English.
- Short code is not automatically good code. CP++ tries to make short code remain semantically explicit through names like `compare`, `Fenwick`, `DSU`, and `Test`.
- The language should support both compressed one-liners and expanded loops because contests require both speed and weird-case flexibility.
- Examples should be treated as style guidance, not as the only allowed idioms.

18. Non-Goals, Tradeoffs, and Open Questions

CP++ is intentionally not a universal programming language. This section records what it should not try to be and which semantics must be nailed down before implementation.

Feature / function	Example	Detailed explanation	Complexity / notes
--------------------	---------	----------------------	--------------------

not a web language	// do not build REST APIs here	CP++ should not chase HTTP, JSON, auth, DOM, React, databases, or cloud deployment as core features. Python/JS/Java are better there.	Non-goal
not a systems language	// use C/C++ for kernels	Although C++-backed, CP++ hides too much memory and allocation detail for operating systems, drivers, embedded, binary exploitation, or performance engineering.	Non-goal
not enterprise OOP	// no FactoryServiceManager	Classes exist for DSU, Trie, SegTree-like helpers, and records with methods. CP++ should not encourage factories, service layers, inheritance-heavy designs, or huge codebases.	Non-goal
sentinel map risk	balance[id] // maybe 0 by default	M[key] returning a sentinel is powerful for CP but dangerous for production semantics. Users should use M.at(key) when absence matters.	Tradeoff
context-sensitive input	List<int> a = input(n);	input() is elegant but implementation-sensitive. The compiler/runtime must clearly infer arity and target type or produce good errors.	Open semantics
copy vs reference loops	for(int x in L) x++;	for(int x in L) likely copies. Mutating iteration requires explicit references. This must be documented to avoid surprises.	Open semantics
floating comparison	Test[x].equals(y); // exact?	Test.equals on floats needs either exact semantics or tolerance support. A future Test[expr].approx(val, eps) may be needed.	Open feature
Map-backed LinkedList update	L.update(5, y);	update semantics are mode-dependent. Normal lists update by index; map-backed lists update by value. This is useful but a footgun.	Document loudly
compare("field") checking	sort(a, compare("socre"));	String field selectors should be compile-time checked when the element type is known. Otherwise CP++ inherits JavaScript typo demons.	Implementation goal
rotation width	x <~ k; // 64-bit width	Circular bit shifts require a width. int uses 64 bits; bitvector uses length; bigint should be invalid unless width is supplied.	Semantic rule
Test.auto generation limits	Test[f,g].auto(1000, valid);	auto() must avoid infinite generation when limit rejects too many candidates. It should report inability to generate enough cases.	Testing rule
performance documentation	// every API lists complexity	Every API should expose its complexity. CP+ + removes ceremony, not algorithmic responsibility.	Core principle

Usage notes

- The non-goals are as important as the features. CP++ should not become a worse Python/JS for web or data tasks.
- Most controversial features are acceptable because CP++ code is usually short-lived. That does not make them safe for production systems.

- Open questions should be resolved before implementation because ambiguous convenience features turn into compiler and debugging pain.
- The language is strongest when it is honest about being an algorithmic DSL with C++-like performance expectations.

Appendix A. Longer CP++ Examples

The following examples show how the individual API rows compose into whole short programs. These are intentionally included to make the reference more usable as a design document.

Two Sum with enumerate and HashMap

```
int n, t = input();
List<int> a = input(n);
HashMap<int,int> seen;

for(int i, x in enumerate(a)) {
    if(t-x in seen) {
        print(seen[t-x], i);
        return;
    }
    seen[x] = i;
}
```

Kruskal MST with string-labeled nodes

```
struct Edge { string u; string v; int w; };

int m = input();
List<Edge> edges;
List<string> nodes;

rep(m) {
    string u, v = input();
    int w = input();
    edges.add({u, v, w});
    nodes.add(u); nodes.add(v);
}

sort(edges, compare("w"));
DSU<string> dsu = DSU(nodes);
int ans = 0;

for(Edge e in edges) {
    if(dsu.merge(e.u, e.v)) ans += e.w;
}
```

```
print(ans);
```

Dynamic range sum queries

```
int n, q = input();
List<int> a = input(n);
Fenwick<int> F = Fenwick(a);
```

```
rep(q) {
    int t = input();
    if(t == 1) {
        int i, x = input();
        F[i] = x;
    } else {
        int l, r = input();
        print(F.query(l, r));
    }
}
```

Brute-vs-fast stress testing

```
pass int brute(List<int> a) {
    int best = -INF;
    for(int l in range(0, len(a))) {
        int s = 0;
        for(int r in range(l, len(a))) {
            s += a[r];
            best = max(best, s);
        }
    }
    return best;
}
```

```
pass int fast(List<int> a) {
    int cur = -INF, best = -INF;
    for(int x in a) {
        cur = max(x, cur + x);
        best = max(best, cur);
    }
    return best;
}
```

```
Test[brute, fast].auto(5000);
```

Appendix B. Complexity Footnotes and Conventions

Convention	Example	Explanation
O(1)*	// see note	Usually amortized or expected constant time. List.add is amortized because reallocation may happen. HashMap operations are expected because collision behavior matters.
Half-open intervals	// see note	Ranges, slices, and query(l,r) use [l,r) unless explicitly stated otherwise. This makes length equal r-l and composes cleanly with prefix sums.
Target-typed input	// see note	input() uses the declared target type and declaration arity. If the compiler cannot infer the target type or number of values, it should report a clear error.
Sentinel values	// see note	Sentinels are defaults, not proof of absence. For maps, use key in M or M.at(key) when absence matters.
Comparators	// see note	compare(1) and compare("field") are intended as common-key comparators. Mixed ascending/descending tie-breaks still require a custom comparator.
Testing	// see note	Test.auto is stress testing, not formal verification. It should find small counterexamples, especially wrong-greedy and edge-case failures.

Appendix C. Full Feature Index

This index repeats every API entry in a compact scan-friendly form. Use it when looking up a feature by name before jumping back to the detailed table above.

Section	Feature	Example	Complexity
1	top-level statements	print("hello world");	Compile-time lowering
1	semicolon rule	int x = 5;	Syntax rule
1	print(x)	print(ans);	O(size of output)
1	print(a, b, c)	print(i, j, dist);	O(size of output)
1	print(x, flush)	print(query, flush);	O(size of output); flush cost
1	describe(x)	describe(dp[i]);	O(size of output)
1	input()	int x = input();	O(length of token)
1	int a, b, c = input()	int a, b, c = input();	O(number of tokens)
1	List<int> L = input()	List<int> L = input();	O(n)
1	List<int> L = input(n)	List<int> a = input(n);	O(n)
1	List^2<int> G = input(n, m)	List^2<int> grid = input(n, m);	O(product of dimensions)
1	string s = input(getline)	string line = input(getline);	O(line length)
1	set_io_in("file.txt")	set_io_in("paint.in");	O(1) setup
1	set_io_out("file.txt")	set_io_out("paint.out");	O(1) setup
1	set_io("paint")	set_io("paint");	O(1) setup
1	f"...{expr}..."	string msg = f"ans:{ans}";	O(result length)
1	{x::2}	print(f"avg:{avg::2}");	O(result length)
1	{x::%}	print(f"rate:{p::%}");	O(result length)
1	{x.round(2)}	print(f"x:{x.round(2)}");	O(result length)
2	bool	bool ok = true;	O(1)
2	int	int n = 200000;	O(1)
2	bigint	bigint x = choose(100,50);	Depends on digit count
2	char	char c = 'A';	O(1)
2	float	float p = 0.125;	O(1)

2	bigfloat	bigfloat z = (bigfloat)x;	Depends on size
2	string	string s = "abc";	Varies
2	bitvector	bitvector seen;	Access may be slower
2	T* pointer	Node<int>* p;	O(1)
2	container defaults	List<int> L;	O(1) or setup
2	int x;	int x;	O(1)
2	int x = 5;	int x = 5;	O(1)
2	int x = input();	int x = input();	O(token length)
2	int a, b, c;	int a, b, c;	O(number of vars)
2	int a, b, c = 1, 2, 3;	int a,b,c = 1,2,3;	O(number of vars)
2	(int) f	int y = (int)avg;	O(1)
2	bigint b = x	bigint b = x;	O(size of x)
2	type promotion	float y = x + 0.5;	Compile-time rule
2	sentinel values	print(M[missing]);	Semantic rule
3	+ - *	int z = a*b + c;	Numeric O(1); list concat O(n+m)
3	/	int q = -3 / 2;	O(1)
3	%	int r = x % mod;	O(1)
3	+= -= *= /= %=	ans += cur;	Varies
3	^ & ~	mask = 1 << b;	O(1) for int
3	x << k, x >> k	int y = x << 2;	O(1) or digit-size
3	x <~ k, x >~ k	int y = x <~ 3;	O(1) int; O(len/word) bitvector
3	x[n]	if(mask[b]) print(b);	O(1) for int
3	bits(x)	bitvector B = bits(mask);	O(number of bits)
3	power2(x, <)	int p = power2(n, <);	O(1) or O(log bits)
3	power2(x, \)	int lb = power2(x, \);	O(1)
3	swap(a, b)	swap(L, R);	O(1) for most containers
3	== != < <= > >=	if(a[i] <= x) {...}	Varies
3	x in C	if(x in seen) print(x);	List O(n); Set O(log n); Hash O(1)*
3	container truthiness	while(Q) process(Q.pop());	O(1) if size stored
3	all(C)	if(all(good)) print("yes");	O(n)
3	any(C)	if(any(hit)) print("found");	O(n)
4	if / else if / else	if(x<0){...} else {...}	Control flow
4	while(condition)	while(Q) {...}	Loop-dependent
4	rep(n)	rep(t) solve();	O(n body)
4	for(int i = 0; i < n; i++)	for(int i=0;i<n;i++){...}	Loop-dependent
4	for(int x in L)	for(int x in a) sum += x;	O(n)
4	range(b)	for(int i in range(n)) {...}	O(1) lazy; O(n) materialized
4	range(a, b)	for(int i in range(l,r)) {...}	O(1) lazy
4	range(a, b, c)	for(int i in range(0,n,2)) {...}	O(1) lazy
4	for(int i, x in enumerate(L))	for(int i, x in enumerate(a)) {...}	O(n)
4	M.items()	for(int x,c in freq.items()) {...}	O(n)
4	M.keys()	List<int> keys = M.keys();	O(n)
4	M.values()	List<int> vals = M.values();	O(n)
4	[expr for int x in C]	List<int> sq = [x*x for int x in a];	O(n)
4	{expr for int x in C}	Set<int> s = {x%m for int x in a};	O(n log n) ordered; O(n)* hash
4	{k:v for int x in C}	Map<int,int> m = {x:x*x for int x in a};	O(n log n) ordered
4	if filter in comprehension	List<int> e = [x for int x in a if x%2==0];	O(n)
4	Gen<T> f(...) { yield x; }	Gen<int> odds(int n){ yield 1; }	Lazy; per-yield cost
4	yield x	yield cur;	Per-yield
5	type name(params) { return x; }	int sq(int x){ return x*x; }	Call-dependent
5	name(params) { ... }	show(int x){ print(x); }	Call-dependent

5	return x;	return ans;	O(1) plus value
5	List<int> L parameter	int f(List<int> L){...}	O(n) copy
5	List<int>& L parameter	void f(List<int>& L){...}	O(1) pass
5	const List<int>& L	int sum(const List<int>& L){...}	O(1) pass
5	pass int f(List<int> L)	pass int f(List<int> L){...}	O(1) container pass
5	const int f(List<int> L)	const int f(List<int> L){...}	O(1) container pass
5	pointer defaults	Node<int>* p;	O(1)
5	inline lambda comparator	sort(a, bool(int x,int y){x>y;});	Comparator-dependent
5	same-interface testing functions	Test[slow, fast].auto(1000);	Compile-time rule
6	Pair<A,B>	Pair<int,string> p = 3,"cat";	O(1)
6	P[0], P[1]	print(P[0], P[1]);	O(1)
6	struct A { ... };	struct Edge{int u,int v,int w;};	Compile-time
6	A a;	Edge e;	O(number of fields)
6	A a = { ... };	Edge e = {u,v,w};	O(number of fields)
6	a.x	print(e.w);	O(1)
6	a[0]	print(e[0]);	O(1)
6	struct comparison	sort(edges);	O(number of fields)
6	class C { ... };	class DS{ List<int> p; };	Compile-time
6	private int x;	private int secret;	Compile-time
6	static int y;	static int count;	O(1) access
6	C(params) { ... }	DSU(int n){ parent=range(n); }	Construction-dependent
6	method definitions	int find(int x){...}	Call-dependent
6	C c = C(params);	Solver s = Solver(n);	Construction-dependent
7	len(C)	int n = len(a);	Usually O(1)
7	C.clear()	seen.clear();	O(n)
7	if(C)	if(L) print(L[-1]);	O(1)
7	C.add(x)	L.add(x);	Container-dependent
7	C.remove()	int x = S.pop();	Container-dependent
7	x in C	if(x in seen) print(x);	Container-dependent
7	min(C), max(C)	print(min(a), max(a));	List O(n); ordered O(1)/O(log n)
7	sum(C)	int total = sum(a);	O(n)
7	C + D	List<int> z = a + b;	List O(n+m); Heap O(n+m)
7	C += D	a += b;	Container-dependent
7	C *= k	s *= 3;	O(k*n)
7	[1,2,3]	List<int> a = [1,2,3];	O(n)
7	{1,2,3}	Set<int> s = {1,2,2,3};	O(n log n) or O(n)*
7	{1:2, 3:4}	Map<int,int> m = {1:2,3:4};	O(n log n) or O(n)*
7	range assigned to container	List<int> a = range(0,n);	O(n)
8	List<T> L;	List<int> L;	O(1)
8	List^k<T>	List^3<int> dp;	Type shorthand
8	L[i]	int last = L[-1];	O(1)
8	L[a:b]	List<int> mid = L[1:-1];	O(length of slice)
8	L.add(x)	L.add(5);	O(1)* amortized
8	L.add(x, i)	L.add(5, 2);	O(n)
8	L.add(x, sorted)	L.add(5, sorted);	O(n)
8	L.remove()	int last = L.remove();	O(1)
8	L.remove(i)	L.remove(3);	O(n)
8	L.index(x)	int i = L.index(7);	O(n)
8	L.index(x, sorted)	int i = L.index(7, sorted);	O(log n)
8	L.lower(x)	int i = L.lower(x);	O(log n)
8	L.upper(x)	int j = L.upper(x);	O(log n)

8	Arr<T> A[size]	Arr<int> a[n];	O(size) init
8	string s;	string s = input();	Varies
8	s[i], s[a:b]	char c = s[0]; string t=s[1:];	O(1) index; O(k) slice
8	string multiplication	string pad = " " * 19;	O(n*k)
8	bitvector B;	bitvector B = bits(mask);	Access O(1) but slower constant
8	B[i] = true	B[3] = true;	O(1)
8	B.count(true)	int ones = B.count(true);	O(words)
9	Set<T> S;	Set<int> S;	Most ops O(log n)
9	Set<T> S(cmp);	Set<List<int>> S(compare(1));	Comparator-dependent
9	S.add(x)	S.add(x);	O(log n)
9	S.delete(x)	S.delete(x);	O(log n)
9	Multiset<T> MS;	Multiset<int> MS;	O(log n) ops
9	MS.count(x)	print(MS.count(x));	O(log n) or O(log n + count)
9	MS.decrement(x)	MS.decrement(x);	O(log n)
9	Map<K,V> M;	Map<int,int> M;	O(log n) key ops
9	Map<K,V> M(cmp);	Map<Node,int> M(compare("id"));	O(log n) ops
9	M[key]	int c = freq[x];	O(log n)
9	M.at(key)	int val = M.at(k);	O(log n)
9	M[key] = value	M[x] = i;	O(log n)
9	M.delete(key)	M.delete(x);	O(log n)
9	key in M	if(x in M) print(M[x]);	O(log n)
9	M.keys()	List<int> keys = M.keys();	O(n)
9	M.values()	List<int> vals = M.values();	O(n)
9	M.items()	for(int x,c in freq.items()) {...}	O(n)
9	HashSet<T> HS;	HashSet<int> HS;	O(1)* expected
9	HashMap<K,V> HM;	HashMap<int,int> HM;	O(1)* expected
9	Counter(V)	Map<int,int> f = Counter(a);	O(n log n) Map; O(n)* HashMap
9	max(M, x)	int k = max(M, x);	O(log n)
10	Heap<T> H;	Heap<int> H;	O(1) construct
10	Heap<T> H(min);	Heap<int> H(min);	O(1) construct
10	Heap<T> H(cmp);	Heap<Node> H(compare("dist"));	Comparator-dependent
10	Heap<T> H = [values]	Heap<int> H = [3,1,4];	O(n)
10	Heap<T> H = Heap(V)	Heap<int> H = Heap(a);	O(n)
10	H.top()	int x = H.top();	O(1)
10	H.pop()	int x = H.pop();	O(log n)
10	H.push(x)	H.push(x);	O(log n)
10	H + G	Heap<int> Z = H + G;	O(n+m)
10	Stack<T> S = [values]	Stack<int> S = [1,2,3];	O(n)
10	S.push(x)	S.push(x);	O(1)*
10	S.pop()	int x = S.pop();	O(1)
10	Queue<T> Q = [values]	Queue<int> Q = [1,2,3];	O(n)
10	Q.push(x)	Q.push(x);	O(1)*
10	Q.pop()	int u = Q.pop();	O(1)
10	Deque<T> D;	Deque<int> D;	O(1) construct
10	D.push_back(x)	D.push_back(x);	O(1)*
10	D.push_front(x)	D.push_front(x);	O(1)*
10	D.pop_back()	int x = D.pop_back();	O(1)
10	D.pop_front()	int x = D.pop_front();	O(1)
11	LinkedList<T> L;	LinkedList<int> L;	O(1) construct
11	LinkedList<T> L = input(n)	LinkedList<int> L = input(n);	O(n)
11	LinkedList<T, circ> L;	LinkedList<int,circ> L;	Node movement wraps

11	LinkedList<T, sing> L;	LinkedList<int,sing> L;	Less memory; no prev
11	LinkedList<T, HashMap> L;	LinkedList<int,HashMap> L;	find O(1)*, add O(1)*
11	LinkedList<T, Map> L;	LinkedList<int,Map> L;	find/add O(log n)
11	LinkedList<T, Map, sing>	LinkedList<int,Map,sing> L;	Mode-dependent
11	len(L)	int n = len(L);	O(1)
11	L.add(x)	L.add(5);	Normal O(1); Hash O(1)*; Map O(log n)
11	L.remove()	int last = L.remove();	O(1)
11	L.remove(N)	L.remove(node);	O(1)
11	L[i]	int last = L[-1];	Doubly O(min(i,n-i)); singly O(i)
11	L.node(i)	Node<int> N = L.node(3);	Doubly O(min(i,n-i)); singly O(i)
11	L.find(x)	Node<int> N = L.find(x);	Hash O(1)*; Map O(log n); normal O(n)
11	N.next()	N = N.next();	O(1)
11	N.next(k)	N = N.next(k);	O(k)
11	N.prev()	N = N.prev();	O(1)
11	N.prev(k)	N = N.prev(k);	O(k)
11	N.val	print(N.val);	O(1)
11	N.update(y)	N.update(y);	Normal O(1); Hash O(1)*; Map O(log n) or more
11	L.update(i, y)	L.update(5, y);	Mode-dependent
12	Graph<T> G;	Graph<int> G;	Construct O(1)
12	WeightedGraph<T,W> W;	WeightedGraph<int,int> W;	Construct O(1)
12	G.add(a,b)	G.add(u, v);	Adjacency-dependent
12	W.add(a,b,c)	W.add(u, v, w);	Adjacency-dependent
12	G.add(a,b,*)	G.add(u, v, *);	Two edge inserts
12	G.addNode(a)	G.addNode(u);	Map/hash dependent
12	G.removeNode(a)	G.removeNode(u);	Potentially O(V+E)
12	G.remove(a,b)	G.remove(u, v);	Adjacency-dependent
12	G.remove(a,b,*)	G.remove(u, v, *);	Two removals
12	G.degree(u)	int d = G.degree(u);	O(1) if stored
12	G.adj(u)	for(int v in G.adj(u)) {...}	O(outdegree)
12	W.adj(u)	for(Pair<int,int> e in W.adj(u)) {...}	O(outdegree)
12	G.vis(u)	G.vis(u);	O(1)* or O(log n)
12	G.seen(u)	if(!G.seen(v)) dfs(v);	O(1)* or O(log n)
12	G.resetVis()	G.resetVis();	O(number visited)
12	no built-in BFS/DFS	// write custom BFS/DFS	Design rule
13	sort(L)	sort(a);	O(n log n)
13	sort(L, reverse)	sort(a, reverse);	O(n log n)
13	sort(L, foo)	sort(a, cmp);	O(n log n * cmp)
13	sort(L, foo, reverse)	sort(a, cmp, reverse);	O(n log n * cmp)
13	compare(i)	sort(rows, compare(1));	Comparator O(1) or field access
13	compare(i,j,k)	sort(rows, compare(0,1,-1));	O(number of keys)
13	compare("field")	sort(edges, compare("w"));	O(field access)
13	compare("a","b")	sort(items, compare("a","b"));	O(number of fields)
13	sort(A, compare(1), reverse)	sort(rows, compare(1), reverse);	O(n log n)
13	reverse(C)	reverse(a);	O(n)
13	L.lower(x)	int i = L.lower(x);	O(log n)
13	L.upper(x)	int j = L.upper(x);	O(log n)
13	first(lo, hi, fun)	int x = first(0,n,ok);	O(log range * fun)
13	last(lo, hi, fun)	int x = last(0,n,ok);	O(log range * fun)
13	prefix(L, op, init)	List<int> p = prefix(a, max, -INF);	O(n)
13	prefix(L, sum)	List<int> p = prefix(a, sum);	O(n)
14	INF	int best = INF;	Constant

14	LLMIN / LLMAX	int low = LLMIN;	Constant
14	MOD	int m = MOD;	Constant
14	MOD1	int mod = MOD1;	Constant
14	abs(x)	int d = abs(a-b);	O(1) or size
14	ceil(x)	int c = ceil(avg);	O(1)
14	round(x)	int r = round(x);	O(1)
14	clamp(x,l,r)	x = clamp(x, 0, n-1);	O(1)
14	pow(a,b)	int p = pow(2, k);	O(log b)
14	pow(a,b,mod)	int p = pow(a, b, MOD);	O(log b)
14	pow(a,-1,mod)	int inv = pow(a, -1, MOD);	O(log mod)
14	div(a,b)	int q = div(a,b);	O(1)
14	div(a,b,mod)	int x = div(a,b,MOD);	O(log mod)
14	gcd(a,b)	int g = gcd(a,b);	O(log min(a,b))
14	lcm(a,b)	int l = lcm(a,b);	O(log min(a,b))
14	choose(n,k)	bigint c = choose(n,k);	Depends on n,k
14	choose(n,k,mod)	int c = choose(n,k,MOD);	Depends on method
14	isPrime(n)	if(isPrime(n)) print(n);	Polylog expected
14	isPrime(n,*)	bool cert = isPrime(n,*);	Polynomial but impractical
14	sieve(n)	List<int> p = sieve(n);	O(n log log n)
14	factor(n)	List<int> d = factor(n);	O(sqrt n) or better
14	prime_factors(n)	Map<int,int> pf = prime_factors(n);	Depends on factoring
14	rand()	float r = rand();	O(1)
14	rand(a,b)	int x = rand(1,n);	O(1)
14	rand(V)	int x = rand(a);	List O(1); set/map may be O(n)
14	seed(x)	seed(1337);	O(1)
14	FFT(A,B)	List<int> C = FFT(A,B);	O(n log n)
14	FFT(A,B,mod)	List<int> C = FFT(A,B,MOD1);	O(n log n)
15	SegTree<T> S = SegTree(n, op, id)	SegTree<int> S = SegTree(n,sum,0);	Build O(n)
15	SegTree<T> S = SegTree(V, op, id)	SegTree<int> S = SegTree(a,min,INF);	O(n)
15	S[i]	int cur = S[i];	O(1) or O(log n)
15	S[i] = x	S[i] = x;	O(log n)
15	S.query(l,r)	print(S.query(l,r));	O(log n)
15	not lazy SegTree	// no range add/assign	Design rule
15	Fenwick<T> F = Fenwick(n)	Fenwick<int> F = Fenwick(n);	O(n) init
15	Fenwick<T> F = Fenwick(V)	Fenwick<int> F = Fenwick(a);	O(n) preferred
15	F.add(i,x)	F.add(i, delta);	O(log n)
15	F[i]	int cur = F[i];	O(log n)
15	F[i] = x	F[i] = x;	O(log n)
15	F.query(l,r)	print(F.query(l,r));	O(log n)
15	DSU<int> D = DSU(n)	DSU<int> D = DSU(n);	O(n) init
15	DSU<T> D = DSU(V)	DSU<string> D = DSU(names);	O(n log n) or O(n)*
15	D.find(x)	int r = D.find(x);	Amortized inverse Ackermann
15	D.merge(u,v)	if(D.merge(u,v)) ans += w;	Amortized inverse Ackermann
15	D.same(x,y)	if(D.same(a,b)) print("yes");	Amortized inverse Ackermann
15	len(D)	print(len(D));	O(1)
15	D.comps()	print(D.comps());	O(1)
15	D.compsize(x)	print(D.compsize(x));	Amortized inverse Ackermann
16	Test t(f1, f2)	Test t(slow, fast);	Setup O(1)
16	t.test({params})	t.test({{1,2,3}});	Cost f1+f2
16	Test[f1,f2].test(params)	Test[slow,fast].test(a);	Cost f1+f2
16	Test[f(params)].equals(val)	Test[gcd(12,18)].equals(6);	Cost f

16	Test[expr].notNull()	Test[p].notNull();	Cost expr
16	Test[expr].isNull()	Test[p].isNull();	Cost expr
16	Test[expr].error(E)	Test[M.at(x)].error(KeyError);	Cost until error
16	Test[expr].greaterThan(x)	Test[ans].greaterThan(0);	Cost expr
16	Test[expr].lessThan(x)	Test(dist).lessThan(INF);	Cost expr
16	Test[f1,f2].auto()	Test[brute,fast].auto();	Generated case count
16	Test[f1,f2].auto(n)	Test[brute,fast].auto(10000);	$n * (f1 + f2)$
16	Test[f1,f2].auto(limit)	Test[brute,fast].auto(valid);	Depends on generation
16	Test[f1,f2].auto(n, limit)	Test[brute,fast].auto(10000, valid);	Depends on generation
16	type-based generation	Test[slow,fast].auto(100);	Design rule
16	failure report	// prints first failing case	$O(\text{size of report})$
16	spiritual TC#2	// small hidden-case generator	Design principle
17	Two Sum pattern	if(t-x in seen) print(seen[t-x],i);	$O(n \log n)$ Map; $O(n)$ * HashMap
17	Kruskal pattern	sort(edges, compare("w"));	$O(m \log m)$
17	Dynamic range sum	F[i]=x; print(F.query(L,r));	$O(\log n)$ per query/update
17	Prefix sums	List<int> P = prefix(a,sum);	Build $O(n)$, query $O(1)$
17	Frequency map	Map<int,int> f = Counter(a);	$O(n \log n)$
17	Sort rows by second column	sort(rows, compare(1));	$O(n \log n)$
17	Interactive output	print(query, flush);	I/O dependent
17	Brute-vs-fast validation	Test[brute,fast].auto(1000);	Testing dependent
17	Circular simulation	cur = cur.next(k);	Depends on k and operations
17	Bitmask DP	if(mask[b]) dp[mask]++;	Bit-dependent
18	not a web language	// do not build REST APIs here	Non-goal
18	not a systems language	// use C/C++ for kernels	Non-goal
18	not enterprise OOP	// no FactoryServiceManager	Non-goal
18	sentinel map risk	balance[id] // maybe 0 by default	Tradeoff
18	context-sensitive input	List<int> a = input(n);	Open semantics
18	copy vs reference loops	for(int x in L) x++;	Open semantics
18	floating comparison	Test[x].equals(y); // exact?	Open feature
18	Map-backed LinkedList update	L.update(5, y);	Document loudly
18	compare("field") checking	sort(a, compare("socre"));	Implementation goal
18	rotation width	x <~ k; // 64-bit width	Semantic rule
18	Test.auto generation limits	Test[f,g].auto(1000, valid);	Testing rule
18	performance documentation	// every API lists complexity	Core principle

End of detailed CP++ reference draft.